LeBlanc

Automated Prompt Enrichment and Iterative Repair for Secure LLM-Generated Code: A Multi-Model Comparative Study

B.Tech Capstone Proposal — Department of Information Technology
KJ Somaiya School of Engineering

Submitted to: Prof. Deepti Patole

Team: Ayushi Ranjan, Swadha Kumari, Vraj Soni, Suryanshu Banerjee — 2025–26

Abstract

LLMs are now routinely used to generate production code, yet empirical studies show that 9–42% of their output contains exploitable security flaws. Existing solutions either retrain the model or evaluate it in a single-turn, single-model setting—neither approach closes the full loop from proactively warning the model, detecting what it still gets wrong, and autonomously fixing it. LeBlanc fills that gap. We build a pipeline of three original engines—automated prompt enrichment, static vulnerability scanning, and iterative repair—that operates around multiple LLMs simultaneously and benchmarks their security output side by side. The result is both a deployable developer tool and a reproducible research dataset that answers four open questions no prior work has addressed together.

The Problem

Developers increasingly rely on LLMs (GPT-4o, Claude, Gemini, DeepSeek) to generate code. These models are trained to prioritise functional correctness, not security. They learn from GitHub repositories saturated with known vulnerabilities—SQL injection (CWE-89), path traversal (CWE-22), unrestricted file upload (CWE-434), hardcoded credentials (CWE-798)—and they reproduce those patterns fluently.

The practical consequence: a developer who asks an LLM to *"write a Flask login with MySQL"* receives working code that is also, with high probability, exploitable. No scanner runs. No warning fires. The vulnerable code ships.

What Existing Research Misses

Our literature review covers eight recent papers. Each makes a genuine contribution, yet each leaves a measurable gap that LeBlanc addresses directly.

Paper	Contribution	Gap we fill
Guiding AI to Fix Its Own Flaws	Multi-model eval + feedback repair	Single-turn only; no automated repair-loop tool
SecuCoGen (Dataset-driven study)	CWE-aware prompts & dataset	180 samples; Python only; no multi-model comparison
CodeSecEval (Is Your Code Safe?)	44-CWE benchmark; auto eval	No enrichment engine; no iterative repair
LPO / DiSCo	Fine-tuning alignment	Requires retraining; not deployable on closed models
Constrained Decoding (CODEGUARD+)	Decode-time security constraints	Requires model internals; not API-accessible
DOMAINEVAL	Multi-domain benchmark	Correctness focus only; no security angle
HexaCoder	Oracle-guided secure fine-tuning	Fine-tune only; not usable on closed models
GitHub Copilot Replication Study	Copilot security audit	Single model; no repair; observational only

Every paper above does at least one of: single-model testing, single-turn evaluation, Python only, or fine-tuning that requires open-weight access. **No paper combines automated CWE-aware prompt enrichment, real static analysis, multi-turn iterative repair, and cross-model benchmarking in one deployable system.** That is the gap LeBlanc fills.

What We Are Building

A developer submits a coding prompt → our system auto-enriches it with CWE warnings → sends it to 3–4 LLMs in parallel → runs each output through real static analysis → autonomously repairs any vulnerabilities in a loop → displays a side-by-side security scorecard showing which model produced the safest code.

Component A — Web Dashboard (React + Flask/FastAPI Backend)

A browser-based interface where the user types any coding prompt, optionally inspects the enriched version, and downloads the cleanest output with a full CWE report. The backend orchestrates all three engines, calls LLM APIs in parallel, and persists every prompt–code–scan–repair triple in a SQLite/PostgreSQL database that doubles as our research dataset.

Component B — VS Code Extension (Phase 2)

An editor plugin that calls the same backend pipeline on the currently open file, highlighting vulnerable lines in-editor and surfacing repair suggestions inline. This extends the platform into an active developer workflow without rebuilding the core.

The Three Engines

LeBlanc calls an LLM at one step of a seven-step pipeline. The LLM is one component— like how a compiler uses a linker. Every step before and after it is original software we build. There are three such engines; none can be replicated by simply forwarding text to an API.

Engine A — Prompt Enrichment

A rule-based local system: a hand-curated `JSON` mapping file of *keyword → CWE IDs → warning text*. Before any LLM call, the engine parses the user's prompt for security-relevant keywords and injects structured warnings automatically. No LLM is involved.

> *Input:* "Write a Flask endpoint for user login with MySQL"
> *Engine A detects:* `database` → CWE-89; `auth` → CWE-521, CWE-798
> *Enriched prompt:* "... IMPORTANT: Avoid CWE-89 (SQL Injection) — use parameterised queries. Avoid CWE-521. Use bcrypt for password hashing."

This approach is grounded in prior work: the CWE-aware prompt strategy in *SecuCoGen* raised GPT-3.5's secure-code rate from 31% to 86%. LeBlanc automates and generalises that strategy across all models.

Engine B — Static Analysis Pipeline

Once the LLM returns code, we run it through Semgrep and Bandit via `subprocess` calls. Their raw JSON output (rule IDs, line numbers, severity) is parsed by a custom mapper into structured reports: CWE ID, severity, line number, plain-English explanation. This is deterministic, rule-based vulnerability detection—not LLM opinion.

Engine C — Iterative Repair Loop

When Engine B finds a vulnerability, Engine C builds a structured repair prompt from the scanner findings and sends it back to the originating LLM. The patched code is immediately re-scanned. The loop runs for up to five iterations or until the scanner returns clean, whichever comes first.

> Attempt 1: LLM generates code → Scanner: 3 vulnerabilities found
> Attempt 2: Repair prompt sent → LLM patches 2 of 3 → Scanner: 1 remaining
> Attempt 3: Repair prompt sent → LLM patches last → Scanner: $\surd$ clean

Every paper in our literature review—*CodeSecEval*, *CODEGUARD+*, the Copilot replication—uses single-turn evaluation only. Nobody has built and measured this loop across models. That is our central research contribution.

Research Questions

We run 100–200 security-sensitive prompts (drawn from SecurityEval, CodeSecEval, and our own curated set) through all models in three modes: (1) plain prompt as baseline, (2) enriched prompt, (3) enriched prompt with repair loop. Every result is auto-logged to the database.

Throughout, *vulnerability rate* is defined as the fraction of prompts for which a given model produces at least one Semgrep or Bandit finding of medium or higher severity, reported separately per model and per CWE category. This yields a clean, reproducible dataset to answer four questions:

- **RQ1 (Enrichment effect):** Does automated CWE-aware prompt enrichment reduce vulnerability rates, and by how much per model?
- **RQ2 (Model comparison):** Which LLM produces the safest code out of the box? Does the ranking change after enrichment?
- **RQ3 (Repair effectiveness):** How many repair iterations does each model need to reach clean code, and is there a ceiling effect?
- **RQ4 (CWE difficulty):** Which CWE types are hardest for LLMs to avoid and hardest to self-repair?

Working paper title:

> *"LeBlanc: Automated Prompt Enrichment and Iterative Repair for Secure LLM-Generated Code — A Multi-Model Comparative Study"*

Technical Architecture

Layer	Technology	Role
Frontend	React / plain HTML+JS	Prompt input, side-by-side viewer, charts
Backend	Python, Flask / FastAPI	Orchestration, LLM calls, repair loop
Engine A	JSON/YAML rule files	Keyword → CWE → warning injection
Engine B	Semgrep, Bandit	Static analysis; CWE-mapped output
Engine C	Python loop controller	Repair prompt builder, re-scan trigger
Database	SQLite / PostgreSQL	Every prompt, code, scan, and iteration
LLM APIs	GPT-4o, Claude, Gemini, DeepSeek	Code generation only
VS Code Ext.	TypeScript + extension API	Editor integration (Phase 2)

Six-Month Timeline

Month	Milestone	Deliverable
Month 1	Setup & skeleton	Semgrep/Bandit running locally; LLM API keys configured; minimal Flask backend that sends a prompt, gets code, and runs a scan end-to-end.
Month 2	Core engines	Engine A (CWE mapping + enrichment), Engine B (scanner parser), Engine C (repair loop) all working in the terminal without a UI.
Month 3	Dashboard & multi-model	React frontend; 3–4 LLMs integrated; side-by-side comparison view; system fully functional end-to-end.
Month 4	Data collection	All 100–200 prompts run in three modes across all models; bugs fixed; full dataset auto-logged to database.
Month 5	Analysis & writing	Statistical analysis, charts, tables; RQ1–RQ4 answered with real numbers; full paper draft complete.
Month 6	Polish & submission	Refined paper; polished dashboard demo; documentation; submission to target venue.

Team Responsibilities

The four team members—Ayushi Ranjan, Swadha Kumari, Vraj Soni, and Suryanshu Banerjee—will divide ownership as follows, with overlap during integration phases.

Area	Scope
Frontend & Dashboard	React interface: prompt input, enrichment toggle, side-by-side code viewer, vulnerability charts, repair-iteration timeline.
Backend & Orchestration	Flask/FastAPI server, parallel LLM API calls, database schema, and the glue connecting all three engines.
Engine A — Prompt Enrichment	CWE mapping database, keyword detection logic, prompt injection templates, and curation of the 100–200 test prompts.
Engines B & C + Research Lead	Semgrep/Bandit integration and output parser (Engine B); iterative repair loop controller and prompt builder (Engine C); data analysis and paper writing from collected results.

Risks and Limitations

Three risks are worth stating upfront. First, the repair loop may not always converge: some vulnerability patterns (e.g., logic flaws outside Semgrep's rule set) may persist across all five iterations. We will report non-convergence rates explicitly rather than treat them as failures, since they are themselves meaningful data about model limits. Second, Semgrep and Bandit produce false positives. We will apply a severity filter (medium or higher only) and manually validate a random 10% sample to estimate false positive rates for the paper. Third, running 200 prompts × 4 models × up to 5 repair iterations generates up to 4 000 LLM calls. We will set a per-experiment API budget cap and use cheaper model tiers for pilot runs before full data collection.

Expected Contributions and Impact

System: A working, open-source tool that any developer can run—type a prompt, get a security-vetted and auto-repaired code output across multiple LLMs with a detailed CWE report. No equivalent tool currently exists.

Research: The first multi-model, multi-mode, iterative-repair evaluation of LLM code security in a single automated pipeline. Prior work is fragmented: *Guiding AI to Fix Its Own Flaws* measures repair offline; *CodeSecEval* benchmarks generation but not repair; *HexaCoder* and *LPO* require fine-tuning inaccessible to most practitioners. LeBlanc provides empirical data across all four research questions using real static analysis on real generated code.

Publication target: We are aiming for venues such as *Empirical Software Engineering* (EMSE), *SANER*, or *MSR*, where the combination of a working system, a reproducible dataset, and cross-model comparative results represents a realistic and meaningful submission.

Conclusion

The gap this project fills is real, well-evidenced by our literature review, and realistically scoped for a four-person, six-month capstone. The system produces original research data automatically—every prompt run logs itself—which means the paper writes itself from the numbers rather than from manual effort.

We are requesting your supervision, periodic feedback on the system design and research methodology, and guidance on the paper submission process. We are committed to regular progress updates at a cadence that suits you.

Submitted by the LeBlanc team *Date:* _____